CLOUD & VIRTUALISATION | TEK-UP UNIVERSITY

Lab 04

Building Docker images: docker build and docker commit

Student
Assil Mhadhbi

Class
Ing1, Groupe C (evening course)

Date
31 October 2026

About this report

In this lab I built Docker images in three different ways: with a Dockerfile (`docker build`), with a multi-stage build that ends on the empty `scratch` image, and from a modified container (`docker commit`). I then pushed an image to a local registry and to Docker Hub, and finally put a small Python script in a container.

Three things are worth saying before the details. First, I worked on my Red Hat virtual machine with Docker Engine 29.9.1. Second, when typing multi-line commands directly in the console went wrong, I wrote my files with `nano` instead. Third, I made several small mistakes along the way (a missing dot, a missing space, a mistyped port). I kept them in the report, in the boxes called Problem I ran into, because each one taught me something about how Docker reads its commands.

The report follows the order of the handout and every part was completed, including the optional Docker Hub part. The Docker Hub access token I used does not appear anywhere in this report.

1. My setup

Item	What I used
Virtual machine	Red Hat Enterprise Linux in VirtualBox, user darkdiva (member of the docker group)
Docker	Docker Engine – Community 29.9.1 (API 1.56), containerd v2.4.2, runc 1.5.2
Image builder	BuildKit (the default builder of this Docker version)
Working folders	~/lab04 for the C program and ~/lab04/python for the Python application
Local registry	The registry:2 image running in a container on port 5000
Docker Hub	Account darkdiva, with a personal access token used instead of the password

2. Checking Docker and preparing the source code

Before starting I checked that Docker works. `docker version` shows the client and the server in the same version, 29.9.1, and the `hello-world` container runs, so the engine, the download from Docker Hub and the creation of containers all work.

```
darkdiva@localhost:~$ docker version
Client: Docker Engine - Community
 Version:           29.9.1
 API version:       1.56
 Go version:        go1.26.9
 Git commit:        dfc520d
 Built:             Wed Oct 14 18:34:49 2026
 OS/Arch:           linux/amd64
 Context:           default

Server: Docker Engine - Community
 Engine:
  Version:          29.9.1
  API version:      1.56 (minimum version 1.40)
  Go version:       go1.26.9
  Git commit:       b26fb77
  Built:            Wed Oct 14 18:31:54 2026
  OS/Arch:          linux/amd64
  Experimental:     false
 containerd:
  Version:          v2.4.2
  GitCommit:        b7a25261e029d68978bbf290df0a62fc819b3c51
 runc:
  Version:          1.5.2
  GitCommit:        v1.5.2-0-g29dd3dc2
 docker-init:
  Version:          0.19.0
  GitCommit:        de40ad0
```

Figure 1. Output of docker version: client and server are both 29.9.1.

```
darkdiva@localhost:~$ docker run --rm hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash
```

Figure 2. The hello-world container runs correctly.

Then I created the working folder and the C program. The command `cat > hello.c << 'EOF'` writes everything that follows, up to the line EOF, into the file. There is no need to install gcc on the VM, because every compilation in this lab happens inside a container.

```
darkdiva@localhost:~$ mkdir -p ~/lab04 && cd ~/lab04
darkdiva@localhost:~/lab04$
```

Figure 3. Creating the working folder ~/lab04.

```
darkdiva@localhost:~/lab04$ cat > hello.c << 'EOF'
> #include <stdio.h>
> int main (void)
> {
> printf("Hello From Docker LAB!\n");
> return 0;
> }
> EOF
```

Figure 4. Writing hello.c with cat and the EOF delimiter.

```
darkdiva@localhost:~/lab04$ cat hello.c
#include <stdio.h>
int main (void)
{
printf("Hello From Docker LAB!\n");
return 0;
}
```

Figure 5. Checking the content of hello.c.

> **Problem I ran into**
>
> My first version of `hello.c` had `#include <stdio.h>` and `int main (void)` on the same line, and a typo in the message. A `#include` must be alone on its line, so gcc would have ignored the rest of that line and the build would have failed. I rewrote the file and checked it again with `cat` before going on.

3. Part 1: an Ubuntu image with docker build (version1)

For the first image I start from Ubuntu 24.04, install gcc, copy the source code, compile it and set the start command. This is my `Dockerfile.ubuntu`:

```
FROM ubuntu:24.04

# Mise à jour + installation + nettoyage dans un seul RUN
RUN apt-get update \
 && apt-get install -y --no-install-recommends gcc libc6-dev \
 && rm -rf /var/lib/apt/lists/*

COPY hello.c /src/hello.c
RUN gcc -o /hello /src/hello.c

CMD ["/hello"]
```

> **Problem I ran into**
>
> This build failed three times before it worked. (1) I forgot the final dot, and Docker answered `requires 1 argument`: the dot is the build context, and without it Docker does not know which folder to send to the engine. (2) The next error said the Dockerfile could not be found, and the log showed that only 2 bytes had been transferred, so the file was empty. I recreated it with `nano`. (3) Docker then refused line 9 with `unknown instruction: CMD["/hello"]`, because I had written `CMD["/hello"]` without a space after `CMD`. I also had `From` in lowercase on the first line, which Docker accepts but reports as a warning, so I changed it to `FROM`.

The command is `docker build -f Dockerfile.ubuntu -t hello:version1 .`: `-f` chooses the Dockerfile, `-t` gives the name and the tag, and the final dot is the build context. The first build took 93.5 seconds because Docker had to download Ubuntu and install gcc.

```
darkdiva@localhost:~/lab04$ nano Dockerfile.ubuntu
darkdiva@localhost:~/lab04$ docker build -f Dockerfile.ubuntu -t hello:version1 .
[+] Building 93.5s (9/9) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile.ubuntu                                                                 0.1s
 => => transferring dockerfile: 311B                                                                                        0.0s
 => WARN: ConsistentInstructionCasing: Command 'From' should match the case of the command majority (uppercase              0.1s
 => [internal] load metadata for docker.io/library/ubuntu:24.04                                                             3.8s
 => [internal] load .dockerignore                                                                                           0.2s
 => => transferring context: 2B                                                                                             0.0s
 => [1/4] FROM docker.io/library/ubuntu:24.04@sha256:fe81560990fae86dc4743202b2cb4d3df98d0bcf949173cecef15ce1              11.0s
 => => resolve docker.io/library/ubuntu:24.04@sha256:fe81560990fae86dc4743202b2cb4d3df98d0bcf949173cecef15ce11              0.1s
 => => sha256:7a48b8f0e5c73d0cc2a85b008978396aa19d253dc1b96fdb0b769ee6dcbeca4a 29.77MB / 29.77MB                            7.0s
 => => extracting sha256:7a48b8f0e5c73d0cc2a85b008978396aa19d253dc1b96fdb0b769ee6dcbeca4a                                    3.5s
 => [internal] load build context                                                                                           0.3s
 => => transferring context: 179B                                                                                           0.0s
 => [2/4] RUN apt-get update && apt-get install -y --no-install-recommends gcc libc6-dev && rm -rf /var/lib/a              44.5s
 => [3/4] COPY hello.c /src/hello.c                                                                                         0.3s
 => [4/4] RUN gcc -o /hello /src/hello.c                                                                                    1.2s
 => exporting to image                                                                                                     31.5s
 => => exporting layers                                                                                                    19.8s
 => => exporting manifest sha256:f381cfdd8e8417504f11c3610307e48486647d473e26b4e2595108ba446a76f8                           0.1s
 => => exporting config sha256:07569ed6d0323a0de3a2517c304e06c1f500b53f80a2a3530f073c0c9708a61d                             0.1s
 => => exporting attestation manifest sha256:d95b1c155bbb2f219255591d85e48af94824c6349083183c2ad032b376b6899e               0.1s
 => => exporting manifest list sha256:60c50212ae1345ead57ae3fa1471e2e18df9b4e7bde25165f97c2626c9a2bb0d                      0.1s
 => => naming to docker.io/library/hello:version1                                                                           0.0s
 => => unpacking to docker.io/library/hello:version1                                                                       11.2s

 1 warning found (use docker --debug to expand):
 - ConsistentInstructionCasing: Command 'From' should match the case of the command majority (uppercase) (line 1)
darkdiva@localhost:~/lab04$ S
```

Figure 6. First build of hello:version1 (93.5 s).

The image appears with 378 MB of disk usage (101 MB of content) and it prints the expected message when I run it.

```
darkdiva@localhost:~/lab04$ docker images hello
                                                                                                  i Info →  U  In Use
IMAGE            ID             DISK USAGE   CONTENT SIZE   EXTRA
hello:version1   581b17f70286        378MB          101MB

darkdiva@localhost:~/lab04$ docker run --rm hello:version1
Hello From Docker LAB!
```

Figure 7. docker images hello and docker run --rm hello:version1.

`docker history` lists the layers. My own layers are the gcc installation (192 MB), the copy of the source (4.1 kB), the compilation (16.4 kB) and the `CMD` (0 B). All the lines below them belong to the Ubuntu base image.

```
darkdiva@localhost:~/lab04$ docker history hello:version1
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
581b17f70286   5 minutes ago   CMD ["/hello"]                                  0B        buildkit.dockerfile.v0
<missing>      5 minutes ago   RUN /bin/sh -c gcc -o /hello /src/hello.c # …   16.4kB    buildkit.dockerfile.v0
<missing>      5 minutes ago   COPY hello.c /src/hello.c # buildkit            4.1kB     buildkit.dockerfile.v0
<missing>      5 minutes ago   RUN /bin/sh -c apt-get update && apt-get ins…   192MB     buildkit.dockerfile.v0
<missing>      5 days ago      /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B
<missing>      5 days ago      /bin/sh -c #(nop) ADD file:eed88e94d6aa7c9da…   85.1MB
<missing>      5 days ago      /bin/sh -c #(nop)  LABEL org.opencontainers.…   0B
<missing>      5 days ago      /bin/sh -c #(nop)  ARG LAUNCHPAD_BUILD_ARCH     0B
<missing>      5 days ago      /bin/sh -c #(nop)  ARG RELEASE                  0B
```

Figure 8. Layers of hello:version1.

I then ran exactly the same build again, with `time` in front. It took 2.3 seconds instead of 93.5, and steps 2/4, 3/4 and 4/4 are marked CACHED: Docker reuses the layers it already built because neither the Dockerfile nor the source file changed.

```
darkdiva@localhost:~/lab04$ time docker build -f Dockerfile.ubuntu -t hello:version1 .
[+] Building 2.3s (9/9) FINISHED                                                                                docker:default
 => [internal] load build definition from Dockerfile.ubuntu                                                              0.1s
 => => transferring dockerfile: 311B                                                                                     0.0s
 => [internal] load metadata for docker.io/library/ubuntu:24.04                                                          1.1s
 => [internal] load .dockerignore                                                                                        0.0s
 => => transferring context: 2B                                                                                          0.0s
 => [1/4] FROM docker.io/library/ubuntu:24.04@sha256:fe81560990fae86dc4743202b2cb4d3df98d0bcf949173cecef15ce11           0.2s
 => => resolve docker.io/library/ubuntu:24.04@sha256:fe81560990fae86dc4743202b2cb4d3df98d0bcf949173cecef15ce11           0.1s
 => [internal] load build context                                                                                        0.0s
 => => transferring context: 87B                                                                                         0.0s
 => CACHED [2/4] RUN apt-get update && apt-get install -y --no-install-recommends gcc libc6-dev && rm -rf /var          0.0s
 => CACHED [3/4] COPY hello.c /src/hello.c                                                                               0.0s
 => CACHED [4/4] RUN gcc -o /hello /src/hello.c                                                                          0.0s
 => exporting to image                                                                                                   0.3s
 => => exporting layers                                                                                                  0.0s
 => => exporting manifest sha256:f381cfdd8e8417504f11c3610307e48486647d473e26b4e2595108ba446a76f8                        0.0s
 => => exporting config sha256:07569ed6d0323a0de3a2517c304e06c1f500b53f80a2a3530f073c0c9708a61d                          0.0s
 => => exporting attestation manifest sha256:38f737de7a7bb928a6403f4e185cb3f82f08164a866cf8a77e2a9cea016bc113            0.1s
 => => exporting manifest list sha256:906a4946a733b58997c1bdc0df15dee37253566fae67bbb2d0c1e05e36d90a0c                   0.1s
 => => naming to docker.io/library/hello:version1                                                                        0.0s
 => => unpacking to docker.io/library/hello:version1                                                                      0.0s

real    0m3.109s
user    0m0.121s
sys     0m0.279s
darkdiva@localhost:~/lab04$
```

Figure 9. Second build, identical: CACHED steps and 2.3 s.

Why are apt-get update and apt-get install in the same RUN? Every RUN creates a layer that Docker can cache. If `apt-get update` is alone, its layer is cached with the package list of that day. If I later add a package in a separate `RUN apt-get install`, Docker reuses the old `update` layer, so the package list is out of date and the install can fail with a 404 error or install old versions. In one RUN, the list is always refreshed together with the install. The `rm -rf /var/lib/apt/lists/*` is in the same RUN for a similar reason: if it were in a later layer, the files would still be stored in the earlier one and the image would not get smaller.

Why do I need libc6-dev in addition to gcc with --no-install-recommends? The header `stdio.h` belongs to the C library and is shipped in the `libc6-dev` package. That package is only a recommended dependency of gcc, so `--no-install-recommends` skips it. Without it the compilation would stop with `stdio.h: No such file or directory`.

Why write ubuntu:24.04 and not just ubuntu? `ubuntu` means `ubuntu:latest`, a tag that changes over time, so the same Dockerfile could give a different image in six months. A precise tag makes the build predictable and reproducible.

4. Part 2: a minimal image with a multi-stage build (version2)

`scratch` is an empty image: no shell and no library. It can only run a static program that carries everything it needs. A multi-stage build lets me compile in a first image (Alpine with gcc) and keep only the executable in the second one. This is my `Dockerfile.scratch`:

```
# Étape 1 : compiler dans une image Alpine qui contient gcc
FROM alpine:3.20 AS build
RUN apk add --no-cache gcc musl-dev
COPY hello.c /src/hello.c
RUN gcc -static -o /hello /src/hello.c

# Étape 2 : image finale vide, on ne copie que l'exécutable
FROM scratch
COPY --from=build /hello /hello
CMD ["/hello"]
```

> **Problem I ran into**
>
> My first build stopped with `pull access denied, repository does not exist`. I had typed `FROM scartch` instead of `FROM scratch`, so Docker looked for an image called `scartch` on Docker Hub. After correcting the letters the build worked.

```
darkdiva@localhost:~/lab04$ nano Dockerfile.scratch
darkdiva@localhost:~/lab04$ docker build -f Dockerfile.scratch  -t hello:version2 .
[+] Building 22.1s (10/10) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile.scratch                                                   0.0s
 => => transferring dockerfile: 296B                                                                           0.0s
 => [internal] load metadata for docker.io/library/alpine:3.20                                                 2.1s
 => [internal] load .dockerignore                                                                              0.0s
 => => transferring context: 2B                                                                                0.0s
 => [build 1/4] FROM docker.io/library/alpine:3.20@sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d  1.6s
 => => resolve docker.io/library/alpine:3.20@sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2  0.1s
 => => sha256:25f1d6b1951ac8eb3740558fe94cb83d377bdadf95fd9f98b50d2e1b96130471 3.63MB / 3.63MB                 0.9s
 => => extracting sha256:25f1d6b1951ac8eb3740558fe94cb83d377bdadf95fd9f98b50d2e1b96130471                       0.5s
 => [internal] load build context                                                                              0.1s
 => => transferring context: 87B                                                                               0.0s
 => [build 2/4] RUN apk add --no-cache gcc musl-dev                                                           15.3s
 => [build 3/4] COPY hello.c /src/hello.c                                                                      0.2s
 => [build 4/4] RUN gcc -static -o /hello /src/hello.c                                                         0.9s
 => [stage-1 1/1] COPY --from=build /hello /hello                                                              0.1s
 => exporting to image                                                                                         0.9s
 => => exporting layers                                                                                        0.3s
 => => exporting manifest sha256:543f2e6b00e422ffe928938c5056091491e2709943f2a1e0b192395adc840d2b              0.1s
 => => exporting config sha256:174fe351bc0c0f7beed7943e166d5d64689577dc4899ddb97eb1c64c2b358138                0.1s
 => => exporting attestation manifest sha256:ec235cc01a55aa7fed48df5972df9f33c6b77db69335fe435aa7aafb714db345  0.1s
 => => exporting manifest list sha256:14b526799968de4873049405051712e6c8eb61cfd7ab1f719f48876fe891369c        0.1s
 => => naming to docker.io/library/hello:version2                                                              0.0s
 => => unpacking to docker.io/library/hello:version2                                                           0.2s
darkdiva@localhost:~/lab04$
```

Figure 10. Build of hello:version2 (22.1 s).

The program runs correctly, and the size is the big surprise: 108 kB for version2, against 378 MB for version1.

```
darkdiva@localhost:~/lab04$ docker run --rm hello:version2
Hello From Docker LAB!

darkdiva@localhost:~/lab04$ docker images hello
IMAGE            ID             DISK USAGE   CONTENT SIZE   EXTRA
hello:version1   906a4946a733       378MB          101MB
hello:version2   14b526799968       108kB         30.1kB
```

Figure 11. Running hello:version2 and comparing the sizes.

Experiment: removing -static. To avoid touching my real file, I copied it to `Dockerfile.test` and removed `-static` from the `gcc` line. The image builds without any error, but it cannot start.

```
darkdiva@localhost:~/lab04$ cp Dockerfile.scratch Dockerfile.test
darkdiva@localhost:~/lab04$ nano Dockerfile.test
darkdiva@localhost:~/lab04$ grep "RUN gcc" Dockerfile.test
RUN gcc -o /hello /src/hello.c
```

Figure 12. Copy of the Dockerfile with -static removed (grep shows the RUN gcc line).

```
darkdiva@localhost:~/lab04$ docker build -f Dockerfile.test -t hello:test .
[+] Building 4.5s (10/10) FINISHED                                                                  docker:default
 => [internal] load build definition from Dockerfile.test                                                    0.0s
 => => transferring dockerfile: 285B                                                                         0.0s
 => [internal] load metadata for docker.io/library/alpine:3.20                                               1.6s
 => [internal] load .dockerignore                                                                            0.1s
 => => transferring context: 2B                                                                              0.0s
 => [build 1/4] FROM docker.io/library/alpine:3.20@sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d  0.1s
 => => resolve docker.io/library/alpine:3.20@sha256:d9e853e87e55526f6b2917df91a2115c36dd7c696a35be12163d44e6e2  0.1s
 => [internal] load build context                                                                            0.1s
 => => transferring context: 87B                                                                             0.0s
 => CACHED [build 2/4] RUN apk add --no-cache gcc musl-dev                                                   0.0s
 => CACHED [build 3/4] COPY hello.c /src/hello.c                                                             0.0s
 => [build 4/4] RUN gcc -o /hello /src/hello.c                                                               0.9s
 => [stage-1 1/1] COPY --from=build /hello /hello                                                            0.1s
 => exporting to image                                                                                       0.9s
 => => exporting layers                                                                                      0.4s
 => => exporting manifest sha256:9afc9c054df6ba101a0a0ec45abb7bb67348857cfa356461cefb177f79e4e62b            0.1s
 => => exporting config sha256:63a29ad204065ed555412c12ead1c8d72f7b5e56da933e36a7f012e35e683c47              0.1s
 => => exporting attestation manifest sha256:f92bfda92e19b0a4ebd43fcd745a3faf640ae6f910197dae7b5f3045e6a23d8f  0.1s
 => => exporting manifest list sha256:fc7e51a84d0740e5225d4bf08df76dfc8e81c17b2e0b17d1899de4917ccc4bb8       0.1s
 => => naming to docker.io/library/hello:test                                                                0.0s
 => => unpacking to docker.io/library/hello:test                                                             0.1s
```

Figure 13. Build of hello:test without -static.

```
darkdiva@localhost:~/lab04$ docker run --rm hello:test
exec /hello: no such file or directory
darkdiva@localhost:~/lab04$ S
```

Figure 14. The error at run time: exec /hello: no such file or directory.

Afterwards I deleted the test file and the test image. My first `docker rmi` failed because I forgot a space (`docker rmihello:test`), which Docker reported as an unknown command.

```
darkdiva@localhost:~/lab04$ rm Dockerfile.test
darkdiva@localhost:~/lab04$ docker rmihello:test
docker: unknown command: docker rmihello:test

Run 'docker --help' for more information
darkdiva@localhost:~/lab04$ docker rmi hello:test
Untagged: hello:test
Deleted: sha256:fc7e51a84d0740e5225d4bf08df76dfc8e81c17b2e0b17d1899de4917ccc4bb8
darkdiva@localhost:~/lab04$ S
```

Figure 15. Cleaning up the experiment.

Trying to open a shell. Both `docker run --rm -it hello:version2 sh` and the same command with `/bin/sh` fail. This is the expected result. With `sh`, runc looks for the program in the folders of the `$PATH` and finds nothing. With `/bin/sh`, the path is complete but the file does not exist. In both cases the command I typed replaces the `CMD ["/hello"]` of the image.

```
darkdiva@localhost:~/lab04$ docker run --rm -it hello:version2 sh
docker: Error response from daemon: failed to create task for container: failed to create shim task: OCI runtime crea
te failed: runc create failed: unable to start container process: error during container init: exec: "sh": executable
 file not found in $PATH

Run 'docker run --help' for more information
darkdiva@localhost:~/lab04$ docker run --rm -it hello:version2 /bin/sh
docker: Error response from daemon: failed to create task for container: failed to create shim task: OCI runtime crea
te failed: runc create failed: unable to start container process: error during container init: exec: "/bin/sh": stat
/bin/sh: no such file or directory

Run 'docker run --help' for more information
```

Figure 16. The two errors with sh and /bin/sh.

To see what is inside the image without entering it, I created a container without starting it, exported its file system and deleted it. The list shows a single real file, `hello` (76,664 bytes), and a few empty items that Docker adds itself (`dev`, `etc`, `proc`, `sys` and `.dockerenv`). There is no `bin` folder.

```
darkdiva@localhost:~/lab04$ docker create --name v2 hello:version2
da9b8dd05203a546e5d9604798f7da11798ecb09dd0d39a4bc2a5f3dcd46d0ca
darkdiva@localhost:~/lab04$ docker export v2 | tar -tv
-rwxr-xr-x 0/0               0 2026-10-31 13:03 .dockerenv
drwxr-xr-x 0/0               0 2026-10-31 13:03 dev/
-rwxr-xr-x 0/0               0 2026-10-31 13:03 dev/console
drwxr-xr-x 0/0               0 2026-10-31 13:03 dev/pts/
drwxr-xr-x 0/0               0 2026-10-31 13:03 dev/shm/
drwxr-xr-x 0/0               0 2026-10-31 13:03 etc/
-rwxr-xr-x 0/0               0 2026-10-31 13:03 etc/hostname
-rwxr-xr-x 0/0               0 2026-10-31 13:03 etc/hosts
lrwxrwxrwx 0/0               0 2026-10-31 13:03 etc/mtab -> /proc/mounts
-rwxr-xr-x 0/0               0 2026-10-31 13:03 etc/resolv.conf
-rwxr-xr-x 0/0           76664 2026-10-31 12:09 hello
drwxr-xr-x 0/0               0 2026-10-31 13:03 proc/
drwxr-xr-x 0/0               0 2026-10-31 13:03 sys/
darkdiva@localhost:~/lab04$ docker rm v2
v2
```

Figure 17. File system of hello:version2 with docker export and tar -tv.

```
darkdiva@localhost:~/lab04$ docker history hello:version2
IMAGE          CREATED          CREATED BY                      SIZE      COMMENT
14b526799968   56 minutes ago   CMD ["/hello"]                  0B        buildkit.dockerfile.v0
<missing>      56 minutes ago   COPY /hello /hello # buildkit   77.8kB    buildkit.dockerfile.v0
darkdiva@localhost:~/lab04$
```

Figure 18. docker history hello:version2: one COPY layer (77.8 kB) and the CMD.

How do the sizes of version1 and version2 compare, and where does the difference come from? Version1 is 378 MB (101 MB of content) and version2 is 108 kB (30.1 kB of content), about 3,500 times smaller. Version1 carries a whole Ubuntu system, gcc and the C library headers (the `apt-get` layer alone is 192 MB). Version2 contains only the static executable: the compiler stayed in the `build` stage, which is not kept.

Why does the experiment without -static fail? Without `-static`, gcc builds a dynamic executable that needs the musl C library and its dynamic loader (`/lib/ld-musl-x86_64.so.1`) when it starts. `scratch` is empty and has no library at all. The message “no such file or directory” is misleading: `/hello` does exist, it is the loader it depends on that is missing. With `-static`, the library code is built into the executable.

Is gcc present in the final image, and how can I check with docker history? No. `docker history hello:version2` shows only two layers, `COPY /hello /hello` (77.8 kB) and `CMD ["/hello"]` (0 B). There is no `apk add` and no gcc layer, because those belong to the build stage that is dropped at the end.

What is a security advantage of an image without a shell? If an attacker took control of the program, there would be no shell and no tool (`ls`, `cat`, `curl`, a package manager) to explore the system or download more tools. The image is also tiny, so there is less software that could contain a vulnerability.

5. Part 3: an image from a container with docker commit (version3)

This time I changed a running container by hand and then took a snapshot of its state. I started an interactive Ubuntu container called `hello_container`, and installed gcc with `apt-get`.

```
darkdiva@localhost:~/lab04$ docker run -it --name hello_container ubuntu:24.04 bash
Unable to find image 'ubuntu:24.04' locally
24.04: Pulling from library/ubuntu
Digest: sha256:fe81560990fae86dc4743202b2cb4d3df98d0bcf949173cecef15ce1127ef852
Status: Downloaded newer image for ubuntu:24.04
root@253f7c984e93:/# apt-get update && apt-get install -y gcc
Get:1 http://archive.ubuntu.com/ubuntu noble InRelease [256 kB]
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [331 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/main amd64 Packages [1808 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2122 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/restricted amd64 Packages [117 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [19.3 MB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [55.6 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1412 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1550 kB]
Get:13 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [2172 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [2240 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Packages [61.7 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1741 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Packages [36.0 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Packages [49.0 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Packages [669 B]
Fetched 33.6 MB in 11s (3048 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu cpp cpp-13 cpp-13-x86-64-linux-gnu cpp-x86-64-linux-gnu
  fontconfig-config fonts-dejavu-core fonts-dejavu-mono gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu
  gcc-x86-64-linux-gnu libaom3 libasan8 libatomic1 libbinutils libbrotli1 libbsd0 libc-dev-bin libc-devtools
  libc6-dev libcc1-0 libcrypt-dev libctf-nobfd0 libctf0 libdeflate0 libexpat1 libfontconfig1 libfreetype6
  libgcc-13-dev libgd3 libgomp1 libgprofng0 libheif-plugin-aomdec libheif-plugin-aomenc libheif1 libhwasan0
  libisl23 libitm1 libjansson4 libjbig0 libjpeg-turbo8 libjpeg8 liblerc4 liblsan0 libmpc3 libmpfr6 libpng16-16t64
  libquadmath0 libsframe1 libsharpyuv0 libtiff6 libtsan2 libubsan1 libwebp7 libx11-6 libx11-data libxau6 libxcb1
  libxdmcp6 libxpm4 linux-libc-dev manpages manpages-dev rpcsvc-proto
Suggested packages:
  binutils-doc gprofng-gui cpp-doc gcc-13-locales cpp-13-doc gcc-multilib make autoconf automake libtool flex bison
  gdb gcc-doc gcc-13-multilib gcc-13-doc gdb-x86-64-linux-gnu glibc-doc libgd-tools libheif-plugin-libde265
  libheif-plugin-x265 libheif-plugin-ffmpegdec libheif-plugin-jpegdec libheif-plugin-jpegenc libheif-plugin-j2kdec
```

Figure 19. Starting hello_container and installing gcc.

Inside the container I created the folder `/hello_directory`, wrote `hello.c` there, compiled it with gcc and ran it. The message “Hello From Docker LAB !” appears, then I left with `exit`.

```
Setting up libbsd0:amd64 (0.12.1-1build1.1) ...
Setting up libcc1-0:amd64 (14.2.0-4ubuntu2~24.04.1) ...
Setting up liblsan0:amd64 (14.2.0-4ubuntu2~24.04.1) ...
Setting up libitm1:amd64 (14.2.0-4ubuntu2~24.04.1) ...
Setting up libctf0:amd64 (2.42-4ubuntu2.10) ...
Setting up libjpeg8:amd64 (8c-2ubuntu11) ...
Setting up manpages-dev (6.7-2) ...
Setting up libxdmcp6:amd64 (1:1.1.3-0ubuntu6) ...
Setting up libxcb1:amd64 (1.15-1ubuntu2) ...
Setting up cpp-13-x86-64-linux-gnu (13.3.0-6ubuntu2~24.04.1) ...
Setting up fontconfig-config (2.15.0-1.1ubuntu2) ...
Setting up libgprofng0:amd64 (2.42-4ubuntu2.10) ...
Setting up libfreetype6:amd64 (2.13.2+dfsg-1ubuntu0.2) ...
Setting up libgcc-13-dev:amd64 (13.3.0-6ubuntu2~24.04.1) ...
Setting up libtiff6:amd64 (4.5.1+git230720-4ubuntu2.5) ...
Setting up libc6-dev:amd64 (2.39-0ubuntu8.9) ...
Setting up libx11-6:amd64 (2:1.8.7-1ubuntu0.1) ...
Setting up libfontconfig1:amd64 (2.15.0-1.1ubuntu2) ...
Setting up binutils-x86-64-linux-gnu (2.42-4ubuntu2.10) ...
Setting up cpp-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up libxpm4:amd64 (1:3.5.17-1ubuntu0.24.04.2) ...
Setting up cpp-13 (13.3.0-6ubuntu2~24.04.1) ...
Setting up gcc-13-x86-64-linux-gnu (13.3.0-6ubuntu2~24.04.1) ...
Setting up binutils (2.42-4ubuntu2.10) ...
Setting up gcc-13 (13.3.0-6ubuntu2~24.04.1) ...
Setting up cpp (4:13.2.0-7ubuntu1) ...
Setting up gcc-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...
Setting up gcc (4:13.2.0-7ubuntu1) ...
Setting up libheif-plugin-aomdec:amd64 (1.17.6-1ubuntu4.9) ...
Setting up libheif1:amd64 (1.17.6-1ubuntu4.9) ...
Setting up libgd3:amd64 (2.3.3-9ubuntu5) ...
Setting up libc-devtools (2.39-0ubuntu8.9) ...
Setting up libheif-plugin-aomenc:amd64 (1.17.6-1ubuntu4.9) ...
Processing triggers for libc-bin (2.39-0ubuntu8.9) ...
root@253f7c984e93:/# mkdir /hello_directory && cd /hello_directory
root@253f7c984e93:/hello_directory# cat > hello.c << 'EOF'
> #include <stdio.h>
> int main (void) {printf("Hello From Docker LAB ! \n"); return 0;}
> EOF
root@253f7c984e93:/hello_directory# gcc -o hello hello.c
root@253f7c984e93:/hello_directory# ./hello
Hello From Docker LAB !
root@253f7c984e93:/hello_directory#
```

Figure 20. Writing, compiling and running hello.c inside the container.

Back on the VM, `docker diff` lists three additions (marked A): the folder `/hello_directory`, the executable `hello` and the source `hello.c`. Then `docker commit` creates the image and sets the default command with `--change`.

> **Problem I ran into**
>
> My first `docker commit` failed with `unknown flag: --change`. I had typed `commmit` with three letters m, so Docker did not know the command and read `--change` as one of its own options. I had also forgotten the name of the new image at the end. The correct command is `docker commit --change 'CMD ["/hello_directory/hello"]' hello_container hello:version3`.

```
root@253f7c984e93:/hello_directory# exit
exit
darkdiva@localhost:~/lab04$ docker diff hello_container | grep hello
A /hello_directory
A /hello_directory/hello
A /hello_directory/hello.c
darkdiva@localhost:~/lab04$ docker commit --change 'CMD ["/hello_directory/hello"]' hello_container hello:version3
sha256:35d273252cba09dd9d5044c976bc44a3fe62ef987489f0865d2a3b01f2b85c58
darkdiva@localhost:~/lab04$
```

Figure 21. exit, docker diff and docker commit.

The image runs correctly. Comparing the two histories is very telling: version1 shows each step of the Dockerfile, but version3 is one anonymous layer of 268 MB whose command is simply `bash`.

```
darkdiva@localhost:~/lab04$ docker run --rm hello:version3
Hello From Docker LAB !
darkdiva@localhost:~/lab04$ docker diff hello_container | grep hello
A /hello_directory
A /hello_directory/hello
A /hello_directory/hello.c
darkdiva@localhost:~/lab04$ docker history hello:version1
IMAGE          CREATED             CREATED BY                                      SIZE      COMMENT
906a4946a733   About an hour ago   CMD ["/hello"]                                  0B        buildkit.dockerfile.v0
<missing>      About an hour ago   RUN /bin/sh -c gcc -o /hello /src/hello.c # …   16.4kB    buildkit.dockerfile.v0
<missing>      About an hour ago   COPY hello.c /src/hello.c # buildkit            4.1kB     buildkit.dockerfile.v0
<missing>      About an hour ago   RUN /bin/sh -c apt-get update && apt-get ins…   192MB     buildkit.dockerfile.v0
<missing>      5 days ago          /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B
<missing>      5 days ago          /bin/sh -c #(nop) ADD file:eed88e94d6aa7c9da…   85.1MB
<missing>      5 days ago          /bin/sh -c #(nop)  LABEL org.opencontainers.…   0B
<missing>      5 days ago          /bin/sh -c #(nop)  ARG LAUNCHPAD_BUILD_ARCH     0B
<missing>      5 days ago          /bin/sh -c #(nop)  ARG RELEASE                  0B
darkdiva@localhost:~/lab04$ docker history hello:version3
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
35d273252cba   2 minutes ago   bash                                            268MB
fe81560990fa   5 days ago      /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B
<missing>      5 days ago      /bin/sh -c #(nop) ADD file:eed88e94d6aa7c9da…   85.1MB
<missing>      5 days ago      /bin/sh -c #(nop)  LABEL org.opencontainers.…   0B
<missing>      5 days ago      /bin/sh -c #(nop)  ARG LAUNCHPAD_BUILD_ARCH     0B
<missing>      5 days ago      /bin/sh -c #(nop)  ARG RELEASE                  0B
darkdiva@localhost:~/lab04$ docker images hello
                                                                  i Info →  U In Use
IMAGE             ID             DISK USAGE   CONTENT SIZE   EXTRA
hello:version1    906a4946a733       378MB          101MB
hello:version2    14b526799968       108kB         30.1kB
hello:version3    35d273252cba       502MB          149MB
darkdiva@localhost:~/lab04$
```

Figure 22. Running version3, docker diff, the histories of version1 and version3, and the sizes of the images.

Why is hello:version3 heavier than hello:version1 although both start from Ubuntu? There are two reasons. I installed gcc with all its recommended packages (no `--no-install-recommends`), so many more packages came in: the layer is 268 MB, against 192 MB for version1. And I never deleted the apt package lists (`/var/lib/apt/lists`), so they stayed in the layer, while in version1 the `rm -rf` runs in the same RUN.

Can I see in docker history hello:version3 which commands I typed in the container? No, the layer only says `bash`. I cannot see which packages were installed or which commands were run. The image is a black box: it cannot be audited, reviewed or rebuilt in the same way, and nobody except its author knows how it was made, which makes maintenance and security updates difficult.

A colleague must rebuild the same image in six months on a newer Ubuntu. Which method makes it easier? The Dockerfile. He only has to change the tag in the FROM line and run the build again, and every step is replayed automatically. With `docker commit` he would have to redo all the manual steps without knowing which ones I did.

6. Comparing the three images

I read the sizes with `docker images hello` and filled in the table of the handout.

Image	Method	Base image	Size	Reproducible?
hello:version1	docker build	ubuntu:24.04	378 MB (content 101 MB)	Yes: the Dockerfile describes every step
hello:version2	docker build, multi-stage	scratch	108 kB (content 30.1 kB)	Yes: Dockerfile and fixed tags
hello:version3	docker commit	ubuntu:24.04	502 MB (content 149 MB)	No: the manual commands are not recorded

From smallest to largest the order is **version2 (108 kB), version1 (378 MB), version3 (502 MB)**. Version2 holds only a static executable on an empty base. Version1 adds a complete Ubuntu system and gcc, but installed in a minimal way and with the apt lists removed. Version3 is the heaviest because gcc came with all its recommended packages and the lists were kept.

7. Publishing the image

7.1 A local registry. A registry is an application too, and it runs in a container. I started the private registry on port 5000 and gave `hello:version2` a name that starts with the address of the registry.

```
darkdiva@localhost:~/lab04$ docker run -d -p 5000:5000 --name registry registry:2
Unable to find image 'registry:2' locally
2: Pulling from library/registry
6d464ea18732: Pull complete
44cf07d57ee4: Pull complete
bbbdd6c6894b: Pull complete
8e82f80af0de: Pull complete
3493bf46cdec: Pull complete
32a76c78501f: Download complete
b537bf6d1146: Download complete
Digest: sha256:a3d8aaa63ed8681a604f1dea0aa03f100d5895b6a58ace528858a7b332415373
Status: Downloaded newer image for registry:2
c72c524478f28764c1c5939834e689903775c8be4497a5a9784d62f47debe260
darkdiva@localhost:~/lab04$ docker tag hello:version2 localhost:5000/hello:2.0
```

Figure 23. Starting registry:2 and tagging the image as localhost:5000/hello:2.0.

> **Problem I ran into**
>
> My first `docker push` failed with `tag does not exist`. I had typed `localhost:5` instead of `localhost:5000`, so Docker looked for an image called `localhost:5/hello:2.0`, which I never created. Once the port was right, the push worked.

After the push, `curl http://localhost:5000/v2/_catalog` answers `{"repositories":["hello"]}`. I then removed the local copies with `docker rmi`, and `docker run` downloaded the image again from my registry before running it. The digest `sha256:14b526799968…` is the same as the identifier of `hello:version2`, so it is the same image.

```
darkdiva@localhost:~/lab04$ docker push localhost:5/hello:2.0
The push refers to repository [localhost:5/hello]
tag does not exist: localhost:5/hello:2.0
darkdiva@localhost:~/lab04$ docker push localhost:5000/hello:2.0
The push refers to repository [localhost:5000/hello]
44136fa355b3: Pushed
66a677e20db7: Pushed
c7ae3fdd4343: Pushed
2.0: digest: sha256:14b526799968de4873049405051712e6c8eb61cfd7ab1f719f48876fe891369c size: 855
darkdiva@localhost:~/lab04$ curl http://localhost:5000/v2/_catalog
{"repositories":["hello"]}
darkdiva@localhost:~/lab04$ docker rmi localhost:5000/hello:2.0 hello:version2
Untagged: localhost:5000/hello:2.0
Untagged: hello:version2
Deleted: sha256:14b526799968de4873049405051712e6c8eb61cfd7ab1f719f48876fe891369c
darkdiva@localhost:~/lab04$ docker run --rm localhost:5000/hello:2.0
Unable to find image 'localhost:5000/hello:2.0' locally
2.0: Pulling from hello
66a677e20db7: Pull complete
44136fa355b3: Already exists
c7ae3fdd4343: Download complete
Digest: sha256:14b526799968de4873049405051712e6c8eb61cfd7ab1f719f48876fe891369c
Status: Downloaded newer image for localhost:5000/hello:2.0
Hello From Docker LAB!
darkdiva@localhost:~/lab04$
```

Figure 24. Push, catalogue, removal of the local copies and run after the download.

7.2 Docker Hub (optional). I created a personal access token in my Docker Hub account (Read & Write) and used it as the password for `docker login -u darkdiva`. Then I tagged the image as `darkdiva/hello:2.0` and pushed it to `docker.io/darkdiva/hello`. The digest is again the same as in the local registry. Docker warns that the credentials are stored unencrypted in `~/.docker/config.json`, so I ran `docker logout` straight after the push.

```
darkdiva@localhost:~/lab04$ docker login -u darkdiva

i Info → A Personal Access Token (PAT) can be used instead.
         To create a PAT, visit https://app.docker.com/settings


Password:

WARNING! Your credentials are stored unencrypted in '/home/darkdiva/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
darkdiva@localhost:~/lab04$ docker tag localhost:5000/hello:2.0 darkdiva/hello:2.0
darkdiva@localhost:~/lab04$ docker push darkdiva/hello:2.0
The push refers to repository [docker.io/darkdiva/hello]
44136fa355b3: Pushed
c7ae3fdd4343: Pushed
66a677e20db7: Pushed
2.0: digest: sha256:14b526799968de4873049405051712e6c8eb61cfd7ab1f719f48876fe891369c size: 855
darkdiva@localhost:~/lab04$
```

Figure 25. Login, tag and push of darkdiva/hello:2.0.

```
darkdiva@localhost:~/lab04$ docker logout
Removing login credentials for https://index.docker.io/v1/
darkdiva@localhost:~/lab04$
```

Figure 26. docker logout removes the saved credentials.

In localhost:5000/hello:2.0, which part is the registry, the repository and the tag? Why is docker push hello:2.0 not enough? The registry is `localhost:5000` (the address of the registry server), the repository is `hello` (the name of the image) and the tag is `2.0` (the version). With `docker push hello:2.0` there is no registry in the name, so Docker assumes the default one, Docker Hub (`docker.io/library/hello`), and would try to send the image there, where it would be refused. The address of the registry has to be part of the name so Docker knows where to send the image.

8. A Python application in a container

In a new folder `~/lab04/python` I wrote `app.py`, a script that prints a message and the version of Python.

```
darkdiva@localhost:~/lab04$ mkdir -p ~/lab04/python && cd ~/lab04/python
darkdiva@localhost:~/lab04/python$ cat > app.py << 'EOF'
> import platform
> print("Hello from Docker Python!")
> print("Version de python :" , platform.python_version())
> EOF
darkdiva@localhost:~/lab04/python$ cat app.py
import platform
print("Hello from Docker Python!")
print("Version de python :" , platform.python_version())
darkdiva@localhost:~/lab04/python$
```

Figure 27. Creating and checking app.py.

The Dockerfile uses the slim variant of the official Python image:

```
# Image officielle Python, variante allégée
FROM python:3.12-slim
# Répertoire de travail dans le conteneur
WORKDIR /app
# Copier le script dans le répertoire de travail
COPY app.py .
# Commande exécutée au démarrage du conteneur
CMD ["python", "app.py"]
```

The slim build took 24.4 seconds, most of it spent downloading the base image.

```
darkdiva@localhost:~/lab04/python$ nano Dockerfile
darkdiva@localhost:~/lab04/python$ docker build -t python-hello:slim .
[+] Building 24.4s (8/8) FINISHED                                                                    docker:default
 => [internal] load build definition from Dockerfile                                                          0.0s
 => => transferring dockerfile: 171B                                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                           4.0s
 => [internal] load .dockerignore                                                                             0.1s
 => => transferring context: 2B                                                                               0.0s
 => [1/3] FROM docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72  16.0s
 => => resolve docker.io/library/python:3.12-slim@sha256:aa943ce411423c7582288f18707ff66eddcbaeb510502d747a72a  0.1s
 => => sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c531531a6f1b057928e 251B / 251B                    0.3s
 => => sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b8ee5a189ed05f63d3 12.12MB / 12.12MB              4.6s
 => => sha256:3eb771151506765 68f88e2957181b8f7e3014e797f1ce8cd640086dfbed45e35 1.29MB / 1.29MB               0.7s
 => => sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5 29.86MB / 29.86MB             13.2s
 => => extracting sha256:6fc242d221a673263eabd14a9ad9584d3177c96e5cb504c891497338e2f912f5                      1.4s
 => => extracting sha256:3eb771151506765 68f88e2957181b8f7e3014e797f1ce8cd640086dfbed45e35                     0.2s
 => => extracting sha256:f574edac59d9f092df596f54e5b52656c803fd3ae30825b8ee5a189ed05f63d3                      0.8s
 => => extracting sha256:acca7542dfc736e9ce20cbefc58b2d09822276d15d4a0c531531a6f1b057928e                      0.0s
 => [internal] load build context                                                                             0.1s
 => => transferring context: 200B                                                                             0.0s
 => [2/3] WORKDIR /app                                                                                        0.5s
 => [3/3] COPY app.py .                                                                                       0.1s
 => exporting to image                                                                                        3.3s
 => => exporting layers                                                                                       0.3s
 => => exporting manifest sha256:9ac22472f038c535576fc00cb268ece415e2316f8554ae63baeea6eb8dcf977f             0.0s
 => => exporting config sha256:eca3d4305d29cfe54c3e89e314a2fb26c7c622b0647741048e17cb5d982ba473               0.0s
 => => exporting attestation manifest sha256:5c5cf31631cddeb4b45e7ac0586650242f632d5bee976bf76c6aae899d5baeed 0.0s
 => => exporting manifest list sha256:62d89bcb6014e2c9f9d5add3fcf93e1ca3e1cfc37d995025283bbe26bede0d52        0.0s
 => => naming to docker.io/library/python-hello:slim                                                          0.0s
 => => unpacking to docker.io/library/python-hello:slim                                                       2.7s
darkdiva@localhost:~/lab04/python$
```

Figure 28. Build of python-hello:slim.

Problem I ran into

For the comparison I changed the first line to FROM `python:3.12`, but my first build of `python-hello:full` gave an image identical to the slim one (same 174 MB). The log showed `load metadata for python:3.12-slim`: the Dockerfile still said `-slim`, and Docker reused its cache with a new tag. After I really changed the first line, the build downloaded the full image and took 250.6 seconds.

```
darkdiva@localhost:~/lab04/python$ nano Dockerfile
darkdiva@localhost:~/lab04/python$ docker build -t python-hello:full .
[+] Building 250.6s (8/8) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 166B                                                                                          0.0s
 => [internal] load metadata for docker.io/library/python:3.12                                                                1.6s
 => [internal] load .dockerignore                                                                                             0.0s
 => => transferring context: 2B                                                                                               0.0s
 => [1/3] FROM docker.io/library/python:3.12@sha256:17b9724493d916f56565c131f389faff2597f46dec6ce09516f95125  152.0s
 => => resolve docker.io/library/python:3.12@sha256:17b9724493d916f56565c131f389faff2597f46dec6ce09516f95125bc  0.1s
 => => sha256:5fa694e7d7469daa72321f8f0d20f5bb54e41ee9a05ab2b675a93abca20fbd5f 251B / 251B                     0.2s
 => => sha256:dc161de3ddb4dc2897a85d767dde501c3d3249351f72ad40286946303c82d315 25.75MB / 25.75MB              31.8s
 => => sha256:7df3eabe14ec411253587a0d60999c086138b697c80bf74bf2f4ba9e61a4174b 6.09MB / 6.09MB                10.6s
 => => sha256:425148f8d5eb4d6dad76e4d080661ed0ad6ed5c270bf028517e6dde8bd0952cc 237.13MB / 237.13MB            85.7s
 => => sha256:c0168c6e6744fc499abc90618818f9e4aaf652a587fb6345b3882dc57de1495c 67.82MB / 67.82MB              95.3s
 => => sha256:7fac3b752bd4f060dcf33c0f54d1fad29fce8ff42b8fb544f9ec95372f0933bf 25.63MB / 25.63MB              40.0s
 => => sha256:a1d156b0aab5fb07d822a7f4681f6d3ed1689073531c34f5668818c233633f11 49.41MB / 49.41MB              62.3s
 => => extracting sha256:a1d156b0aab5fb07d822a7f4681f6d3ed1689073531c34f5668818c233633f11                      2.6s
 => => extracting sha256:7fac3b752bd4f060dcf33c0f54d1fad29fce8ff42b8fb544f9ec95372f0933bf                      1.2s
 => => extracting sha256:c0168c6e6744fc499abc90618818f9e4aaf652a587fb6345b3882dc57de1495c                      3.6s
 => => extracting sha256:425148f8d5eb4d6dad76e4d080661ed0ad6ed5c270bf028517e6dde8bd0952cc                     48.6s
 => => extracting sha256:7df3eabe14ec411253587a0d60999c086138b697c80bf74bf2f4ba9e61a4174b                      0.4s
 => => extracting sha256:dc161de3ddb4dc2897a85d767dde501c3d3249351f72ad40286946303c82d315                      1.0s
 => => extracting sha256:5fa694e7d7469daa72321f8f0d20f5bb54e41ee9a05ab2b675a93abca20fbd5f                      0.0s
 => [internal] load build context                                                                             0.0s
 => => transferring context: 86B                                                                              0.0s
 => [2/3] WORKDIR /app                                                                                        0.7s
 => [3/3] COPY app.py .                                                                                       0.1s
 => exporting to image                                                                                       95.9s
 => => exporting layers                                                                                       0.3s
 => => exporting manifest sha256:254209608c9856325b0d66c54a5c5b2cb732acd90e140e8a9c0b802c420844a0             0.0s
 => => exporting config sha256:06d45222792edb4bb73cb7198d0de105db5aa56e78f3f9adb7bc939df095a8ba               0.0s
 => => exporting attestation manifest sha256:85af657c0b8d78e08ae4170d4ceecf1d5d088daf5c1df7a99d1a95ca25e6ce8d  0.0s
 => => exporting manifest list sha256:69a24d6d26d25544af2aaa45f71fae41f45553b0f49ad8c78674c61b97cd52ed         0.0s
 => => naming to docker.io/library/python-hello:full                                                          0.0s
 => => unpacking to docker.io/library/python-hello:full                                                      95.4s
```

Figure 29. Build of python-hello:full from python:3.12.

The full image is 1.59 GB against 174 MB for slim. I also overrode the default command when starting the container: `python -c "print(2 + 2)"` prints 4, because a command given at the end of `docker run` replaces the CMD of the Dockerfile.

```
darkdiva@localhost:~/lab04/python$ docker images python-hello
                                                                                                  i Info →  U In Use
IMAGE                ID             DISK USAGE   CONTENT SIZE   EXTRA
python-hello:full    69a24d6d26d2       1.59GB          412MB
python-hello:slim    62d89bcb6014        174MB         43.3MB
darkdiva@localhost:~/lab04/python$ docker run --rm python-hello:slim python -c "print(2 + 2)"
4
darkdiva@localhost:~/lab04/python$ S
```

Figure 30. Sizes of the two python-hello images and the run with another command.

Image	Disk usage	Content size
python-hello:slim	174 MB	43.3 MB
python-hello:full	1.59 GB	412 MB

What is the size difference between slim and full, and what does the full image contain in addition? The full image is about 9 times bigger, roughly 1.4 GB more. It is built on a complete Debian system with development tools (gcc, make, library headers, git and other packages), whereas the slim image only holds what is needed to run Python.

What happened to the CMD of the Dockerfile in step 5? It was replaced. A command written after the image name in `docker run` takes priority over the `CMD`, so `python -c "print(2 + 2)"` ran instead of `python app.py`.

Why did the 2025 version of the lab use python:3.9, and why should it be changed now? Python 3.9 was a common, current version at that time. It reached its end of life in October 2025, so it no longer receives security fixes. Today I use a version that is still maintained (3.12 here).

9. Cleaning up and final questions

Before cleaning, `docker system df` showed 3 containers (268.1 MB, all reclaimable), 11 images (3.334 GB) and 2.31 GB of build cache. I removed the containers `hello_container` and `registry` with `docker rm -f`, which freed about 268 MB. `docker image prune` removes only dangling images, so it freed 0 B: I had none. About 3.2 GB of images are still reclaimable, but they are unused, not dangling, so this command leaves them alone.

```
darkdiva@localhost:~/lab04/python$ docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          11        3         3.334GB   3.181GB (95%)
Containers      3         1         268.1MB   268.1MB (100%)
Local Volumes   1         1         31.63kB   0B (0%)
Build Cache     32        0         2.31GB    169.3MB
darkdiva@localhost:~/lab04/python$ docker rm -f hello_container registry
hello_container
registry
darkdiva@localhost:~/lab04/python$ docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
darkdiva@localhost:~/lab04/python$ docker  system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          11        1         3.334GB   3.219GB (96%)
Containers      1         0         0B        0B
Local Volumes   1         0         31.63kB   31.63kB (100%)
Build Cache     32        0         2.31GB    169.3MB
darkdiva@localhost:~/lab04/python$
```

Figure 31. docker system df before and after the cleanup, and docker image prune.

Rank hello:version1, version2 and version3 from smallest to largest and explain each gap. Version2 (108 kB), then version1 (378 MB), then version3 (502 MB). Version2 holds only a static executable. Version1 adds a full Ubuntu system and gcc, but installed with `--no-install-recommends` and with the apt lists removed. Version3 is the largest because gcc came with all its recommended packages and the lists stayed in the layer.

Give two reasons to prefer a Dockerfile to docker commit in a team. First, reproducibility and traceability: the Dockerfile describes every step, while a committed image is a black box with one `bash` layer. Second, teamwork and maintenance: a Dockerfile is a text file that can be versioned with Git, reviewed and updated by changing a tag, it is easy to automate, and it benefits from the cache and gives smaller images.

When is docker commit still useful? For troubleshooting and exploring: saving the state of a broken container so I can study it, testing a change quickly before writing it in a Dockerfile, or keeping a trace of a state to investigate. It is not a production method.

What is the role of the dot at the end of docker build, and what would happen if I ran the command from my home folder? The dot is the build context, the folder that is sent to the Docker engine and where `COPY` looks for files. Without it Docker answers `requires 1 argument`, which I met during this lab. From my home folder, the whole home folder would be sent to the engine (slow and pointless), the Dockerfile given with `-f` would not be found, and `COPY hello.c` would fail because `hello.c` is not at the root of that context.

What does a multi-stage build bring compared with compiling on the host, as in the 2025 version of the lab? No compiler is needed on the VM: the compilation happens in a container with the same toolchain for everyone, so the result is reproducible on any machine. The final image keeps only the executable and leaves the build tools in the intermediate stage, so it is tiny and safer. All of it is described in a single Dockerfile.

10. Summary

Task	Result	Comment
Docker check and source code	Done	Docker 29.9.1; hello.c rewritten after a mistake
Part 1: image with docker build	Done	378 MB; rebuild in 2.3 s thanks to the cache
Part 2: multi-stage scratch image	Done	108 kB; the build without -static fails as expected
Part 3: docker commit	Done	502 MB; a single anonymous layer
Local registry	Done	Push, catalogue and pull checked
Docker Hub (optional)	Done	darkdiva/hello:2.0 pushed, same digest as the local copy
Python application	Done	slim 174 MB against full 1.59 GB
Cleanup and final questions	Done	About 268 MB freed by removing the two containers

11. What I learned

The biggest lesson was how much a Dockerfile gives compared with a container I change by hand. The same program gave three images of 378 MB, 108 kB and 502 MB, and only the two built from a Dockerfile can be explained and rebuilt. The cache also became concrete when the same build went from 93.5 seconds to 2.3 seconds.

The multi-stage build showed me that a small image is not only about disk space. An image on `scratch` has no shell and no compiler, and the error without `-static` taught me that a program can fail because of a file it needs and that is not in the image, even when the message talks about the program itself.

Most of my errors came from details: a missing dot, a missing space after `CMD`, a letter swapped in `scratch`, a port written as 5 instead of 5000, and a tag changed while the Dockerfile still said `-slim`. I learned to read

the error message first and to check what Docker actually did, for example the line `load metadata for` in the build log, instead of trusting that the command I typed was the command I meant.